

Shashwath Ks Assignment 2 Day 02

Programming Essentials For Ai

Dated : 11/04/2026

AI-12 Python Assignment

Source-W3Schools

Topics Covered
Python Operators
Python Lists
Python Tuples
Python Sets
Python Dictionaries

Python operators

Python operators are functional symbols that trigger specific logical, arithmetic, or bitwise computations on operands. While the `+` operator performs arithmetic addition for numeric types, it is polymorphic, meaning its behavior changes such as performing concatenation-based on the data types of the variables involved.

```
# Example for arithmetic operators
print(22 + 5) # Addition
print(15 - 5) # Subtraction
print(4 * 4) # Multiplication
print( 5 / 4) # Division
print( 5 // 3) # Floor Division returns the rounded-up value of the division operation
```

Output Block

```
27
10
16
1.25
1
```

```
sum1 = 100 + 50      # 150 (100 + 50)
sum2 = sum1 + 250    # 400 (150 + 250)
sum3 = sum2 + sum2   # 800 (400 + 400)
print(sum1, sum2, sum3)
```

Arithmetic Operators

Arithmetic operators are used with numeric values to perform common mathematical operations:

Operator	Name	Example
+	Addition	x + y
-	Subtraction	x - y
*	Multiplication	x * y
/	Division	x / y
%	Modulus	x % y
**	Exponentiation	x ** y
//	Floor division	x // y

```
# Other different Operations on two variables
x = 15
y = 4

print(x % y) # Modulus operator returns the remainder of the division
print(x ** y) # Power operator returns the result of x ^ y
print(x // y) # Floor division operator returns the integer part of the division.
```

Output Block

```
3
50625
3
```

Assignment Operators

Assignment operators are used to assign values to variables:

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3
&=	x &= 3	x = x & 3
=	x = 3	x = x 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3
:=	print(x := 3)	x = 3, print(x)

```
# Assignment operations
x = 10
y = 20

x +=5
print(x)

x -=9
print(x)
```

Output Block

```
15
6
```

The Walrus Operator

the := operator, known as the "walrus operator". It assigns values to variables as part of a larger expression.

In simpler terms, it lets you say: "I want to calculate this thing, save it to a variable, and also use it right here in this line of code."

```
numbers = [1, 2, 3, 4, 5] # List of numbers

if (count := len(numbers)) > 3: # Walrus operator to assign len of list
    print(f"List has {count} elements")
```

Output Block

```
List has 5 elements
```

```
while (user_input := input("Enter something (or 'quit'): ")) != "quit":  
    print(f"You typed: {user_input}")
```

Output Block

```
You typed: Shash  
You typed: Ash  
You typed: qui
```

Comparison Operators

Comparison operators are used to compare two values:

Operator	Name	Example
==	Equal	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Comparison operators return True or False based on the comparison:

Chaining Comparison Operators

Python allows you to chain comparison operators:

```
x = 5  
  
print(1 < x < 10) # Chained comparison.  
  
print(1 < x and x < 10) # This is the equivalent of the chained comparison above, but  
it is less concise and less readable.
```

Output Block

```
True  
True
```

Logical Operators

Logical operators are used to combine conditional statements:

Operator	Description	Example
and	Returns True if both statements are true	<code>x < 5 and x < 10</code>
or	Returns True if one of the statements is true	<code>x < 5 or x < 4</code>
not	Reverses the result (False if the result is true)	<code>not(x < 5 and x < 10)</code>

```
x = 5
print(x > 0 and x < 10) # This is the equivalent of the chained comparison above, but
it is less concise and less readable.
print(x < 5 or x > 10) # Test if a number is less than 5 or greater than 10.
print(not(x > 3 and x < 10)) # Reverse the result with not.
```

Output Block

```
True
False
False
```

Identity Operators

Identity operators are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location:

Operator	Description	Example
is	Returns True if both variables are the same object	<code>x is y</code>
is not	Returns True if variables are not the same object	<code>x is not y</code>

Difference Between `is` and `==`

- `is` - Checks if both variables point to the same object in memory
- `==` - Checks if the values of both variables are equal

```
# The is operator returns True if both variables point to the same object
x = ["apple", "banana"]
y = ["apple", "banana"]
z = x

print(x is z) # Returns True because z is the same object as x
print(x is y) # Returns False because y is a different object from x
print(x == y) # Returns True because the contents of x and y are the same
```

Output Block

```
True
False
True
```

Membership Operators

Membership operators are used to test if a sequence is presented in an object

Operator	Description	Example
in	Returns True if a value is present in the sequence	x in y
not in	Returns True if a value is not present in the sequence	x not in y

```
# Membership operators script to test if a value is present in a list or not
fruits = ["apple", "banana", "cherry"]

print("banana" in fruits) # prints True if present in the list.
print("pineapple" not in fruits) # prints True if not present in the list.

text = "Hello World" # A string variable to test the membership operators

print("H" in text) # prints True if "H" is present in the string.
print("hello" in text) # prints True if "hello" is present in the string.
print("z" not in text) # prints True if "z" is not present in the string.
```

Output Block

```
True
True
True
False
True
```

Bitwise Operators

Bitwise operators are used to compare (binary) numbers.

Operator	Name	Description	Example
&	AND	Sets each bit to 1 if both bits are 1	x & y
	OR	Sets each bit to 1 if one of two bits is 1	x y
^	XOR	Sets each bit to 1 if only one of two bits is 1	x ^ y
~	NOT	Inverts all the bits	~x
<<	Zero fill left shift	Shifts left, pushing zeros from the right and dropping leftmost bits	x << 2
>>	Signed right shift	Shifts right, copying leftmost bit and dropping rightmost bits	x >> 2

```
# Bitwise operators examples
print (5 & 3) # Bitwise AND
print (5 | 3) # Bitwise OR
print (5 ^ 3) # Bitwise XOR
print (~5) # Bitwise NOT
```

Output Block

```
1
7
6
-6
```

Operator Precedence

Operator precedence describes the order in which operations are performed.

Precedence Order

The precedence order is described in the table below, starting with the highest precedence at the top.

Operator	Description
()	Parentheses
**	Exponentiation
+x, -x, ~x	Unary plus, unary minus, and bitwise NOT
*, /, //, %	Multiplication, division, floor division, modulus
+, -	Addition and subtraction
<<, >>	Bitwise left and right shifts
&	Bitwise AND
^	Bitwise XOR
	Bitwise OR
==, !=, >, >=, <, <=, is, is not, in, not in	Comparisons, identity, and membership operators
not	Logical NOT
and	AND
or	OR

If two operators have the same precedence, the expression is evaluated from left to right.

```
# Addition + and subtraction - has the same precedence, and therefore we evaluate the
expression from left to right:
```

```
print(5 + 4 - 7 + 3)
print((5+4)-(7+3))
```

Output Block

```
5
-1
```

List

Lists are used to store multiple items within a single variable.

In Python, lists are one of the four built-in data types used to store collections of data. The other three are Tuple, Set, and Dictionary—each with its own unique characteristics and use cases.

Lists are created using square brackets

```
# Create a list
thislist = ["apple", "banana", "cherry"] # list is created with 3 items
print(thislist)
```

List Items

List items are ordered, changeable, and allow duplicate values.

List items are indexed, where the first item has index `[0]`, the second item has index `[1]`, and so on.

Ordered

When we say that lists are ordered, it means that the items have a defined sequence, and that sequence does not change.

If you add new items to a list, they will be placed at the end of the list.

Changeable

Lists are changeable, meaning that items can be modified, added, or removed after the list has been created.

Allow Duplicates

Since lists are indexed, they can contain multiple items with the same value.

```
# This example shows that the list allows for the duplicate values
thislist = ["apple", "banana", "cherry", "apple", "cherry"]
print(thislist)
```

Output Block

```
['apple', 'banana', 'cherry', 'apple', 'cherry']
```

List Length

To determine how many items a list has, use the `len()` function:

```
thislist = ["apple", "banana", "cherry"]
print(len(thislist)) # Returns the length of the list
```

Output Block

```
3
```

List Items - Data Types

List items can be of any data type, meaning a single list can store elements such as integers, strings, floats, booleans, or even other lists.

Lists can also contain a mix of different data types within the same list, making them highly flexible for storing diverse collections of data.

```
list1 = ["apple", "banana", "cherry"]
list2 = [1, 5, 7, 9, 3]
list3 = [True, False, False]
print(f"{list1}, {list2}, {list3}")
```

Output Block

```
['apple', 'banana', 'cherry'], [1, 5, 7, 9, 3], [True, False, False]
```

List with different values

```
#A list with strings, integers and boolean values:
list1 = ["abc", 34, True, 40, "male"]
print(list1)

# returning the type of the list
print(type(list1))
```

Output Block

```
['abc', 34, True, 40, 'male']
<class 'list'>
```

The list() Constructor

It is also possible to create a new list using the `list()` constructor.

The `list()` constructor provides an alternative way to initialize a list, and it can be used to create lists from other iterable objects.

```
## Use of list constructor to create a list from the tuple
thislist = list(("apple", "banana", "cherry")) # note the double round-brackets
print(thislist)
```

Output Block

```
['apple', 'banana', 'cherry']
```

Python Collections (Arrays)

There are four built-in collection data types in Python:

- **List:** A collection that is ordered and changeable, and allows duplicate members.
- **Tuple:** A collection that is ordered and unchangeable, and allows duplicate members.
- **Set:** A collection that is unordered, unchangeable*, and unindexed, and does not allow duplicate members.
- **Dictionary:** A collection that is ordered** and changeable, and does not allow duplicate members.

Access Items

List items are indexed and you can access them by referring to the index number:

```
## Accessing the list elements
# Print the second item of the list:

thislist = ["apple", "banana", "cherry"]
print(thislist[1])
```

Output Block

banana

Negative Indexing

Negative indexing means start from the end

-1 refers to the last item, **-2** refers to the second last item etc.

```
thislist = ["apple", "banana", "cherry"]
print(thislist[-1]) # Return the last element in the list
```

Output Block

cherry

Range of Indexes

You can specify a range of indexes by specifying where to start and where to end the range.

When specifying a range, the return value will be a new list with the specified items.

```
# Returns the elements from the list starting from the 2nd position to the 4 position
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print(thislist[2:5])

## Returns the items from the beginning to, but NOT including, "kiwi"
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print(thislist[:4])

### Returns the items from "cherry" to the end
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print(thislist[2:])
```

Output Block

```
['cherry', 'orange', 'kiwi']
['apple', 'banana', 'cherry', 'orange']
['cherry', 'orange', 'kiwi', 'melon', 'mango']
```

Range of Negative Indexes

Specify negative indexes if you want to start the search from the end of the list.

```
# This example returns the items from "orange" (-4) to, but NOT including "mango" (-1)
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print(thislist[-4:-1])
```

Output Block

```
['orange', 'kiwi', 'melon']
```

Check if Item Exists

To determine if a specified item is present in a list use the in keyword.

```
thislist = ["apple", "banana", "cherry"]
if "apple" in thislist:
    print("Yes, 'apple' is in the fruits list")
```

Output Block

```
Yes, 'apple' is in the fruits list
```

Python - Change List Items

Change Item Value

To change the value of a specific item, refer to the index number.

```
# Changing the value in the list
thislist = ["apple", "banana", "cherry"]
thislist[1] = "blackcurrant"
print(thislist)
```

Output Block

```
['apple', 'blackcurrant', 'cherry']
```

Change a Range of Item Values

To change the value of items within a specific range, define a list with the new values, and refer to the range of index numbers where you want to insert the new values.

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]
# Changing the value of the list from the positions 1(in list) to te 2nd value.
thislist[1:3] = ["blackcurrant", "watermelon"]
print(thislist)
```

```
# If you insert more items than you replace, the new items will be inserted where you specified, and the remaining items will move accordingly
```

```
thislist = ["apple", "banana", "cherry"]  
# Change the second value by replacing it with two new values  
thislist[1:2] = ["blackcurrant", "watermelon"]  
print(thislist)  
  
# Change the second and third value by replacing it with one value  
thislist[1:3] = ["watermelon"]  
print(thislist)
```

Output Block

```
['apple', 'blackcurrant', 'watermelon', 'orange', 'kiwi', 'mango']  
['apple', 'blackcurrant', 'watermelon', 'cherry']  
['apple', 'watermelon', 'cherry']
```

Insert Items

To add a new item to a list without replacing any existing values, you can use the `insert()` method.

The `insert()` method adds an item at a specified index.

```
thislist = ["apple", "banana", "cherry"]  
# Inserting the value 'Watermelon' in the 2nd position (3rd)  
thislist.insert(2, "watermelon")  
print(thislist)
```

Output Block

```
['apple', 'banana', 'watermelon', 'cherry']
```

Append Items

To add an item to the end of a list, use the `append()` method.

```
thislist = ["apple", "banana", "cherry"]  
## Appending the value 'orange' at the end of the list  
thislist.append("orange")  
print(thislist)
```

Output Block

```
['apple', 'banana', 'cherry', 'orange']
```

Insert Items

To insert a list item at a specified index, use the `insert()` method.

The `insert()` method inserts an item at the specified index.

```
thislist = ["apple", "banana", "cherry"]
# Insert the 'orange' in the second position
thislist.insert(1, "orange")
print(thislist)
```

Output Block

```
['apple', 'orange', 'banana', 'cherry']
```

Extend List

To add elements from another list to the current list, use the `extend()` method.

```
thislist = ["apple", "banana", "cherry"]
tropical = ["mango", "pineapple", "papaya"]
# Extending the list named thislist with the list called tropical
thislist.extend(tropical)
print(thislist)
```

Output Block

```
['apple', 'banana', 'cherry', 'mango', 'pineapple', 'papaya']
```

Add Any Iterable

The `extend()` method is not limited to lists; it can be used to add elements from any iterable object, such as tuples, sets, or dictionaries.

```
thislist = ["apple", "banana", "cherry"]
thistuple = ("kiwi", "orange")
# The following code will extend the list with the tuple variable
thislist.extend(thistuple)
print(thislist)
```

Output Block

```
['apple', 'banana', 'cherry', 'kiwi', 'orange']
```

Python - Remove List Items

Remove Specified Item

The `remove()` method deletes the first occurrence of the specified item from a list.

```
print(thislist)
# Remove the list Item Banana
thislist.remove("banana")
print(thislist)
```

Output Block

```
['apple', 'banana', 'cherry', 'kiwi', 'orange']
['apple', 'cherry', 'kiwi', 'orange']
```

If there are more than one item with the specified value, the `remove()` method removes the first occurrence

```
# Duplicate entries of the banana is defined
thislist = ["apple", "banana", "cherry", "banana", "kiwi"]
#removes the first occurrence of "banana"
thislist.remove("banana")
print(thislist)
```

Output Block

```
['apple', 'cherry', 'banana', 'kiwi']
```

Remove Specified Index

The `pop()` method removes the item at the specified index.

```
thislist = ["apple", "banana", "cherry"]
# Removes the element in the position 1 of the list i.e banana
thislist.pop(1)
print(thislist)

thislist = ["apple", "banana", "cherry"]
# without mentioning the position in the `pop` parameter it removes the last entry.
thislist.pop()
print(thislist)
```

Output Block

```
['apple', 'cherry']
['apple', 'banana']
```

The `del` keyword also removes the specified index

```
thislist = ["apple", "banana", "cherry"]
# deletes the first element in the list
del thislist[0]
print(thislist)

# The del keyword can also delete the list completely.
del thislist
```

```
# to confirm the list is present but the variable deleted and released from the memory.  
print(thislist)
```

Output Block

```
['banana', 'cherry']
```

```
NameError: name 'thislist' is not defined
```

Clear the List

The `clear()` method removes all items from a list.

The list still exists, but it becomes empty.

```
thislist = ["apple", "banana", "cherry"]  
print("Before", thislist)  
# Empty the list without removing the variable from the memory  
thislist.clear()  
print("After", thislist)
```

Output Block

```
Before ['apple', 'banana', 'cherry']
```

```
After []
```

Python - Loop Lists

Loop Through a List

You can iterate through the items in a list using a `for` loop.

```
thislist = ["apple", "banana", "cherry"]  
# Printing all the values in the list assigning x as a variable to print one single value for each iteration  
for x in thislist:  
    print(x)
```

Output Block

```
apple  
banana  
cherry
```

Loop Through the Index Numbers

You can also iterate through list items by referring to their index numbers.

To do this, use the `range()` and `len()` functions to create a suitable iterable.

```
thislist = ["apple", "banana", "cherry"]
# iterate through the entire list with mentioning the len of the list
for i in range(len(thislist)):
    print(thislist[i])
```

Output Block

```
apple
banana
cherry
```

Using a While Loop

You can iterate through list items using a `while` loop.

Use the `len()` function to determine the length of the list, then start at index `0` and access each item by referring to its index position.

Remember to increment the index by `1` after each iteration.

```
thislist = ["apple", "banana", "cherry"]
i = 0
while i < len(thislist):
    print(thislist[i])
    i+=1
```

Output Block

```
apple
banana
cherry
```

Looping Using List Comprehension

List comprehension provides a shorter and more concise syntax for iterating through lists.

```
thislist = ["apple", "orange", "banana"]
y = [x for x in thislist]
print(y)
```

Output Block

```
['apple', 'orange', 'banana']
```

Python - List Comprehension

List Comprehension

List comprehension provides a shorter syntax for creating a new list based on the values of an existing list.

Example

For example, given a list of fruits, you may want to create a new list that contains only the fruits with the letter "a" in their names.

Without list comprehension, you would need to use a `for` loop along with a conditional statement.

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
newlist = [] # new empty list initialized

for x in fruits:
    if "a" in x:
        newlist.append(x) # append items to the list with value containing string 'a'

print(newlist)
```

Output Block

```
['apple', 'banana', 'mango']
```

```
# Shorter and one-line method using list comprehension

fruits = ["apple", "banana", "cherry", "kiwi", "mango"]

newlist = [x for x in fruits if "a" in x]

print(newlist)
```

Output Block

```
['apple', 'banana', 'mango']
```

The Syntax

```
newlist = [expression for item in iterable if condition == True]
```

The result is a new list, while the original list remains unchanged.

Condition

The condition acts as a filter, allowing only the items that evaluate to `True` to be included in the new list.

```
newlist = [x for x in fruits if x != "apple"]
print(newlist)
```

Output Block

```
['banana', 'cherry', 'kiwi', 'mango']
```

The condition `x != "apple"` will return `True` for all elements other than "apple", making the new list contain all fruits except "apple".

Example

Without an `if` statement:

```
newlist = [x for x in fruits]
```

Iterable

The iterable can be any iterable object, such as a list, tuple, or set.

Example

You can also use the `range()` function to create an iterable:

```
newlist = [x for x in range(10)]
```

Expression

The expression represents the current item in the iteration, and it also defines the final value that will be included in the new list.

You can modify or manipulate this expression before it is added as an item in the resulting list.

```
# Converts all the list items in list 'fruits' and converts them to uppercase
newlist = [x.upper() for x in fruits]
print(newlist)
```

Output Block

```
['APPLE', 'BANANA', 'CHERRY', 'KIWI', 'MANGO']
```

```
# Set all values in the new list to 'hello':
newlist = ['hello' for x in fruits]
print(newlist)
```

Output Block

```
['hello', 'hello', 'hello', 'hello', 'hello']
```

The expression can also contain conditions, not like a filter, but as a way to manipulate the outcome:

```
# Assigns values to newlist excluding 'banana' and 'orange'
newlist = [x if x != "banana" else "orange" for x in fruits]
print(newlist)
```

Output Block

```
['apple', 'orange', 'cherry', 'kiwi', 'mango']
```

Python - Sort Lists

Sort List Alphanumerically

List objects have a `sort()` method that sorts the list alphanumerically in ascending order by default.

```
# Sorting the defined list 'thislist' of type string
print(f"The list before sorting : {thislist}")
thislist.sort()
print(f"The list after sorting : {thislist}")
```

Output Block

```
The list before sorting : ['apple', 'orange', 'banana']
The list after sorting : ['apple', 'banana', 'orange']
```

```
# Sorting the list numerically
thislist = [100, 50, 65, 82, 23]
thislist.sort()
print(thislist)
```

Output Block

```
[23, 50, 65, 82, 100]
```

Sort Descending

To sort descending, use the keyword argument `reverse = True`

```
# Sorting the list in the descending order
thislist = ["orange", "mango", "kiwi", "pineapple", "banana"]
thislist.sort(reverse = True)
print(thislist)

# For numerical
numList = [100, 50, 65, 82, 23]
numList.sort(reverse = True)
print(numList)
```

Output Block

```
['pineapple', 'orange', 'mango', 'kiwi', 'banana']  
[100, 82, 65, 50, 23]
```

Customize Sort Function

You can also customize your own function by using the keyword argument `key = function`.

The function will return a number that will be used to sort the list (the lowest number first).

```
# Script to call a function and sort the elements in the list  
def myfunc(n):  
    return abs(n - 50)  
  
thislist = [100, 50, 65, 82, 23]  
thislist.sort(key = myfunc)  
print(thislist)
```

Output Block

```
[50, 65, 23, 82, 100]
```

Case Insensitive Sort

By default, the `sort()` method is case sensitive, which means that all uppercase letters are sorted before lowercase letters.

```
# the regular sort with case-sensitive approach  
thislist = ["banana", "Orange", "Kiwi", "cherry"]  
thislist.sort()  
print(thislist)
```

Output Block

```
['Kiwi', 'Orange', 'banana', 'cherry']
```

```
# Performing a case-insensitive sort of the list  
  
thislist = ["banana", "Orange", "Kiwi", "cherry"]  
thislist.sort(key = str.lower)  
print(thislist)
```

Output Block

```
['banana', 'cherry', 'Kiwi', 'Orange']
```

Reverse Order

If you want to reverse the order of a list, regardless of alphabetical or numeric order, you can use the `reverse()` method.

The `reverse()` method reverses the current order of the elements in the list.

```
# Reverse order of the list without case-sensitiveness
thislist = ["banana", "Orange", "Kiwi", "cherry"]
thislist.reverse()
print(thislist)
```

Output Block

```
['cherry', 'Kiwi', 'Orange', 'banana']
```

Python - Copy Lists

Copy a List

You cannot copy a list by simply assigning it using `list2 = list1`, because `list2` will only be a reference to `list1`, and any changes made in `list1` will also affect `list2`.

Using the `copy()` Method

You can use the built-in `copy()` method to create an actual copy of a list.

```
thislist = ["apple", "banana", "cherry"]
mylist = thislist.copy() # It's a shallow copy object of the original list 'thislist'
print(mylist)
```

Output Block

```
['apple', 'banana', 'cherry']
```

Use the `list()` method

Another way to make a copy is to use the built-in method `list()`.

```
# Create a copy of a list using list constructor.
thislist = ["apple", "banana", "cherry"]
mylist = list(thislist)
print(mylist)
```

Output Block

```
['apple', 'banana', 'cherry']
```

Use the slice Operator

You can also make a copy of a list by using the : (slice) operator.

```
# Make a copy of a list with the : operator:

thislist = ["apple", "banana", "cherry"]
mylist = thislist[:]
print(mylist)
```

Output Block

```
['apple', 'banana', 'cherry']
```

Python - Join Lists

Join Two Lists

There are several ways to join, or concatenate, two or more lists in Python.

One of the simplest ways is using the `+` operator.

```
list1 = ["a", "b", "c"]
list2 = [1, 2, 3]
# appending a list using `+` operators.
list3 = list1 + list2
print(list3)
```

Output Block

```
['a', 'b', 'c', 1, 2, 3]
```

```
# Append list2 into list1

list1 = ["a", "b", "c"]
list2 = [1, 2, 3]

for x in list2:
    list1.append(x)

print(list1)
```

Output Block

```
['a', 'b', 'c', 1, 2, 3]
```

```
# Use the extend() method to add list2 at the end of list1
```

```
list1 = ["a", "b" , "c"]
list2 = [1, 2, 3]

list1.extend(list2)
print(list1)
```

Output Block

```
['a', 'b', 'c', 1, 2, 3]
```

Python - List Methods

List Methods

Python provides a set of built-in methods that can be used with lists.

Method	Description
<code>append()</code>	Adds an element at the end of the list
<code>clear()</code>	Removes all elements from the list
<code>copy()</code>	Returns a copy of the list
<code>count()</code>	Returns the number of elements with the specified value
<code>extend()</code>	Adds elements from another list (or any iterable) to the end of the current list
<code>index()</code>	Returns the index of the first occurrence of the specified value
<code>insert()</code>	Adds an element at a specified position
<code>pop()</code>	Removes the element at the specified position
<code>remove()</code>	Removes the item with the specified value
<code>reverse()</code>	Reverses the order of the list
<code>sort()</code>	Sorts the list

Python Tuples

```
mytuple = ("apple", "banana", "cherry")
```

Tuple

Tuples are used to store multiple items in a single variable.

Tuple is one of the four built-in data types in Python used to store collections of data. The other three are List, Set, and Dictionary—each with different characteristics and use cases.

A tuple is a collection that is ordered and unchangeable.

Tuples are written using round brackets `()`.

```
# Creating a tuple and displaying the items in tuple
thistuple = ("apple", "banana", "cherry")
print(thistuple)
```

Output Block

```
('apple', 'banana', 'cherry')
```

Tuple Items

Tuple items are ordered, unchangeable, and allow duplicate values.

Tuple items are indexed, where the first item has index `[0]`, the second item has index `[1]`, and so on.

Ordered

When we say tuples are ordered, it means the items have a defined sequence, and that sequence does not change.

Unchangeable

Tuples are unchangeable, meaning items cannot be changed, added, or removed after the tuple is created.

Allow Duplicates

Since tuples are indexed, they can contain multiple items with the same value.

```
# Tuples allow duplicate values:
# In this tuple there is two duplicate values apple and cherry.
thistuple = ("apple", "banana", "cherry", "apple", "cherry")
print(thistuple)
```

Output Block

```
('apple', 'banana', 'cherry', 'apple', 'cherry')
```

Tuple Length

To determine how many items a tuple has, use the `len()` function.

```
# Count of items present in the tuple

thistuple = ("apple", "banana", "cherry")
print(len(thistuple))
```

Output Block

```
3
```

Create Tuple With One Item

To create a tuple with only one item, you have to add a comma after the item, otherwise Python will not recognize it as a tuple.

```
# Creating a tuple with just one item
thistuple = ("apple",) # Comma is required otherwise the type would be string.
print(type(thistuple))

#NOT a tuple would be of type str
thistuple = ("apple")
print(type(thistuple))
```

Output Block

```
<class 'tuple'>
<class 'str'>
```

Tuple Items - Data Types

Tuple items can be of any data type, meaning a tuple can store values such as integers, strings, floats, booleans, or other objects.

A tuple can also contain different data types at the same time, making it flexible for storing mixed collections of data.

```
tuple1 = ("apple", "banana", "cherry") # Tuple of type String
tuple2 = (1, 5, 7, 9, 3) # Tuple of type int
tuple3 = (True, False, False) # Tuple of type Boolean

# A tuple with strings, integers and boolean values:

tuple1 = ("abc", 34, True, 40, "male")

print(type(tuple1))
```

Output Block

```
<class 'tuple'>
```

The tuple() Constructor

It is also possible to create a tuple using the `tuple()` constructor.

The `tuple()` constructor provides an alternative way to initialize a tuple from other iterable objects.

```
thistuple = tuple(("apple", "banana", "cherry")) # note the double round-brackets
print(thistuple)
```

Output Block

```
('apple', 'banana', 'cherry')
```

Python Collections (Arrays)

There are four built-in collection data types in the Python programming language:

- **List:** A collection that is ordered and changeable, and allows duplicate members.
- **Tuple:** A collection that is ordered and unchangeable, and allows duplicate members.
- **Set:** A collection that is unordered, unchangeable*, and unindexed, and does not allow duplicate members.
- **Dictionary:** A collection that is ordered** and changeable, and does not allow duplicate members.

When choosing a collection type, it is important to understand its properties. Selecting the right type for a given dataset can help preserve meaning and improve efficiency or security.

Access Tuple Items

You can access tuple items by referring to the index number, inside square brackets

```
# Create a tuple containing some fruit names
fruit_tuple = ("apple", "banana", "cherry")

# Access and print the second item
print(fruit_tuple[1]) # This works just like list indexing
```

Output Block

```
banana
```

Negative Indexing

Negative indexing means starting from the end of a sequence.

-1 refers to the last item, **-2** refers to the second last item, and so on.

```
# Similar negative indexing will return the last element in the tuple
print(thistuple[-1])
```

Output Block

```
cherry
```

Range of Indexes

You can define a range of indexes by indicating both the starting and ending positions.

When a range is specified, the result is a new tuple that contains only the elements within that range.

```
# Create a tuple with multiple fruit names
fruit_tuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
# Slice the tuple to get the 3rd, 4th, and 5th items
selected_fruits = fruit_tuple[2:5]
# Print the selected items
print(selected_fruits) # This returns a new tuple with the sliced values
```

Output Block

```
('cherry', 'orange', 'kiwi')
```

Range of Negative Indexes

Specify negative indexes if you want to start the search from the end of the tuple.

```
fruit_tuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
# Use negative indexing to slice the tuple
subset_fruits = fruit_tuple[-4:-1]
print(subset_fruits) # Returns a tuple with the selected elements
```

Output Block

```
('orange', 'kiwi', 'melon')
```

Check if Item Exists

To check whether a particular element is present in a tuple, use the `in` keyword.

```
fruit_tuple = ("apple", "banana", "cherry")
# Check if "apple" exists in the tuple
if "apple" in fruit_tuple:
    print("Yes, 'apple' is in the fruits tuple")
```

Output Block

```
Yes, 'apple' is in the fruits tuple
```

Python : Update Tuples

Tuples are immutable, which means their elements cannot be modified, added, or removed after creation.

However, there are some alternative approaches to work around this limitation.

Change Tuple Values

Once a tuple has been created, its values cannot be altered. This property is known as immutability.

A common workaround is to convert the tuple into a list, make the necessary changes to the list, and then convert it back into a tuple.

```
fruit_tuple = ("apple", "banana", "cherry")
# Convert tuple to list to modify it
fruit_list = list(fruit_tuple)
fruit_list[1] = "kiwi"
# Convert list back to tuple
fruit_tuple = tuple(fruit_list)
print(fruit_tuple) # Updated tuple
```

Output Block

```
('apple', 'kiwi', 'cherry')
```

Add Items

Since tuples are immutable, they do not support an `append()` method. However, there are alternative ways to include new elements.

- 1 **Convert to a list:** Similar to modifying a tuple, you can transform it into a list, add the desired item(s), and then convert it back into a tuple.

```
fruit_tuple = ("apple", "banana", "cherry")
# Convert tuple to list to modify it
fruit_list = list(fruit_tuple)
# Add a new item to the list
fruit_list.append("orange")
# Convert list back to tuple
fruit_tuple = tuple(fruit_list)
```

- 1 Add tuple to a tuple. You are allowed to add tuples to tuples, so if you want to add one item, (or many), create a new tuple with the item(s), and add it to the existing tuple.

```
# Create a single-item tuple
extra_fruit = ("orange",)
# Add the new tuple to the existing tuple
fruit_tuple += extra_fruit
print(fruit_tuple)
```

Output Block

```
('apple', 'banana', 'cherry', 'orange', 'orange')
```

Remove Items

Note: You cannot directly remove items from a tuple.

Since tuples are immutable, their elements cannot be deleted. However, you can use the same workaround as used for modifying and adding items: convert the tuple into a list, perform the removal, and then convert it back into a tuple.

```
fruit_tuple = ("apple", "banana", "cherry")
# Convert tuple to list to modify it
fruit_list = list(fruit_tuple)
# Remove "apple" from the list
fruit_list.remove("apple")
# Convert list back to tuple
fruit_tuple = tuple(fruit_list)
print(fruit_tuple)
```

Output Block

```
('banana', 'cherry')
```

```
# The del keyword can delete the tuple completely.
fruit_tuple = ("apple", "banana", "cherry")
del fruit_tuple
print(fruit_tuple) #this will raise an error because the tuple no longer exists
```

Output Block

```
NameError: name 'fruit_tuple' is not defined
```

Unpacking a Tuple

When a tuple is created, values are typically assigned to it. This process is known as **packing** a tuple.

```
# Packing values into a tuple
fruits = ("apple", "banana", "cherry")
print(fruits)
```

Output Block

```
('apple', 'banana', 'cherry')
```

But, in Python, we are also allowed to extract the values back into variables. This is called "unpacking".

```
fruits = ("apple", "banana", "cherry")
# Unpacking tuple values into variables
green, yellow, red = fruits
print(green)
print(yellow)
print(red)
```

Output Block

```
apple  
banana  
cherry
```

Using Asterisk (*)

If the number of variables is fewer than the number of values in a tuple, you can use an asterisk (*) before a variable name. The remaining values will then be collected into that variable as a list.

```
fruits = ("apple", "banana", "cherry", "strawberry", "raspberry")  
# Unpack first two values, rest go into a list  
green, yellow, *red = fruits  
print(green)  
print(yellow)  
print(red)
```

Output Block

```
apple  
banana  
['cherry', 'strawberry', 'raspberry']
```

If the asterisk is added to another variable name than the last, Python will assign values to the variable until the number of values left matches the number of variables left.

```
fruits = ("apple", "mango", "papaya", "pineapple", "cherry")  
# Unpack first and last item, middle items go into a list  
green, *tropic, red = fruits  
print(green)  
print(tropic)  
print(red)
```

Output Block

```
apple  
['mango', 'papaya', 'pineapple']  
cherry
```

Python - Loop Tuples

Loop Through a Tuple

You can iterate over the items in a tuple using a `for` loop.

```
# Iterate through the items and print the values:  
thistuple = ("apple", "banana", "cherry")  
for x in thistuple:  
    print(x)
```

Output Block

```
apple  
banana  
cherry
```

Loop Through Index Numbers

You can also iterate through a tuple by using index positions.

To do this, use the `range()` and `len()` functions to generate a valid sequence of indexes for looping.

```
# Print all items by referring to their index number.  
thistuple = ("apple", "banana", "cherry")  
for i in range(len(thistuple)):  
    print(thistuple[i])
```

Output Block

```
apple  
banana  
cherry
```

Using a While Loop

You can iterate through tuple elements using a `while` loop.

Use the `len()` function to get the total number of items in the tuple. Start from index `0` and access each element using its index position.

Make sure to increment the index by `1` after each iteration to avoid an infinite loop.

```
# Print all items, using a while loop to go through all the index numbers:  
thistuple = ("apple", "banana", "cherry")  
i = 0  
while i < len(thistuple):  
    print(thistuple[i])  
    i = i + 1
```

Output Block

```
apple  
banana  
cherry
```

Python - Join Tuples

Join Two Tuples

You can combine two or more tuples using the `+` operator.

```
# Join two tuples:
tuple1 = ("a", "b" , "c")
tuple2 = (1, 2, 3)
# Combining two tuples 'tuple1' and 'tuple2' and store it in 'tuple3'
tuple3 = tuple1 + tuple2
print(tuple3)
```

Output Block

```
('a', 'b', 'c', 1, 2, 3)
```

Multiply Tuples

If you want to multiply the content of a tuple a given number of times, you can use the `*` operator.

```
# Multiply the fruits tuple by 2.
fruits = ("apple", "banana", "cherry")
mytuple = fruits * 2 # Tuple items are multiplied and stored in new variable mytuple
print(mytuple)
```

Output Block

```
('apple', 'banana', 'cherry', 'apple', 'banana', 'cherry')
```

Python - Tuple Methods

Python provides two built-in methods for working with tuples.

Method	Description
<code>count()</code>	Returns how many times a specified value appears in a tuple
<code>index()</code>	Searches for a specified value in the tuple and returns its position

Python Sets

```
myset = {"apple", "banana", "cherry"}
```

Set

Sets are used to store multiple items in a single variable.

A set is one of the four built-in data types in Python used to store collections of data. The other three are List, Tuple, and Dictionary—each with different properties and use cases.

A set is a collection that is unordered, unchangeable*, and unindexed.

Sets are written with curly brackets.

```
# Creating a Set

thisset = {"apple", "banana", "cherry"}
print(thisset)
```

Output Block

```
{'cherry', 'apple', 'banana'}
```

Set Items

Set items are unordered, unchangeable, and do not allow duplicate values.

Unordered

Unordered means that set items do not have a defined sequence.

Set items may appear in a different order each time they are used, and they cannot be accessed using an index or key.

Unchangeable

Set items are unchangeable, meaning they cannot be modified after the set has been created.

Duplicates Not Allowed

Sets cannot have two items with the same value.

```
# Duplicate values will be ignored:

thisset = {"apple", "banana", "cherry", "apple"} # Apple is entered two time the set
will only print out disticnt values

print(thisset)
```

Output Block

```
{'cherry', 'apple', 'banana'}
```

```
# Here in this script the True and 1 is considered the same value:
thisset = {"apple", "banana", "cherry", True, 1, 2}
print(thisset)

# False and 0 are considered as a single value
thisset = {"apple", "banana", "cherry", False, True, 0}
print(thisset)
```

Output Block

```
{True, 2, 'cherry', 'apple', 'banana'}  
{False, True, 'cherry', 'apple', 'banana'}
```

Get the Length of a Set

To determine how many items a set has, use the `len()` function.

```
# Get the number of items in a set:  
thisset = {"apple", "banana", "cherry"}  
print(len(thisset))
```

Output Block

```
3
```

Set Items - Data Types

Set items can be of any data type.

```
set1 = {"apple", "banana", "cherry"} # Type str  
set2 = {1, 5, 7, 9, 3} # Type int  
set3 = {True, False, False} # Type bool  
  
# A set with strings, integers and boolean values:  
set1 = {"abc", 34, True, 40, "male"}  
  
print(type(set1))
```

Output Block

```
<class 'set'>
```

The set() Constructor

It is also possible to use the `set()` constructor to make a set.

```
# Using the set() constructor to make a set from a tuple  
  
thisset = set(("apple", "banana", "cherry")) # note the double round-brackets  
print(thisset)
```

Output Block

```
{'cherry', 'apple', 'banana'}
```

Access Items

You cannot access items in a set using an index or a key.

However, you can iterate through set items using a `for` loop, or check whether a specific value exists in a set using the `in` keyword.

```
# Loop through the set, and print the values:
thisset = {"apple", "banana", "cherry"}
for x in thisset:
    print(x)
```

Output Block

```
cherry
apple
banana
```

```
# Check if "banana" is present in the set:
thisset = {"apple", "banana", "cherry"}
print("banana" in thisset)
```

Output Block

```
True
```

```
# Check if "banana" is NOT present in the set:
thisset = {"apple", "banana", "cherry"}
print("banana" not in thisset)
```

Output Block

```
False
```

Python - Add Set Items

Add Items

You can add items to a set using the `add()` method.

```
print(thisset)
# Add a item to the set using add() function
thisset.add("orange")
print(thisset)
```

Output Block

```
{'orange', 'cherry', 'apple', 'banana'}  
{'orange', 'cherry', 'apple', 'banana'}
```

Add Sets

To add items from another set into the current set, use the `update()` method.

```
# Add elements from tropical into thisset:  
thisset = {"apple", "banana", "cherry"}  
tropical = {"pineapple", "mango", "papaya"}  
# Adds the contents of set tropical to the 'thisset'  
thisset.update(tropical)  
print(thisset)
```

Output Block

```
{'mango', 'pineapple', 'papaya', 'cherry', 'apple', 'banana'}
```

Add Any Iterable

The object passed to the `update()` method does not have to be a set. It can be any iterable object, such as a list, tuple, or dictionary.

```
thisset = {"apple", "banana", "cherry"} #set  
mylist = ["kiwi", "orange"] #list  
# Adding the contents of the list to the end of the list  
thisset.update(mylist)  
print(thisset)
```

Output Block

```
{'orange', 'cherry', 'kiwi', 'apple', 'banana'}
```

Python - Remove Set Items

Remove Item

To remove an item from a set, you can use the `remove()` or `discard()` methods.

```
# Remove "banana" by using the remove() method:  
thisset = {"apple", "banana", "cherry"}  
print(thisset)  
thisset.remove("banana") # removing an item using 'remove()'  
print(thisset)
```

Output Block

```
{'cherry', 'apple', 'banana'}
{'cherry', 'apple'}
```

```
# Remove "banana" by using the discard() method:
thisset = {"apple", "banana", "cherry"}
print(thisset)
thisset.discard("banana") # removing an item using 'discard()'
print(thisset)
```

Output Block

```
{'cherry', 'apple', 'banana'}
{'cherry', 'apple'}
```

You can also use the `pop()` method to remove an item. However, this method removes an item based on its index, and if no index is specified, it removes the last item.

The `pop()` method also returns the removed item as its output.

```
fruit_set = {"apple", "banana", "cherry"}
# Remove and return a random item from the set
removed_item = fruit_set.pop()
print(removed_item) # Shows the removed item
print(fruit_set)    # Remaining items in the set
```

Output Block

```
cherry
{'banana', 'apple'}
```

Note: Sets are unordered, so when using the `pop()` method, you do not know which item that gets removed.

```
# The clear() method empties the set:
thisset = {"apple", "banana", "cherry"}
# Empties the set but variable reference still present in memory
thisset.clear()
print(thisset)
```

Output Block

```
set()
```

```
# The del keyword will delete the set completely:
thisset = {"apple", "banana", "cherry"}
del thisset # Deletes the set and variable reference in memory.
print(thisset)
```

Python - Loop Sets

Loop Items

You can iterate over the elements of a set using a `for` loop.

Since sets are unordered collections, the loop accesses each item one by one without any specific sequence.

```
# Loop through the set, and print the values:
thisset = {"apple", "banana", "cherry"}
# Iterating using a for loop for each record
for x in thisset:
    print(x)
```

Output Block

```
cherry
banana
apple
```

Python - Join Sets

Join Sets

In Python, there are multiple ways to combine two or more sets.

The `union()` and `update()` methods are used to merge all elements from the sets.

The `intersection()` method returns only the common elements between sets.

The `difference()` method returns elements that are present in the first set but not in the other set(s).

The `symmetric_difference()` method returns all elements except those that appear in both sets.

Union

The `union()` method creates and returns a new set containing all elements from both sets.

```
set_a = {"a", "b", "c"}
set_b = {1, 2, 3}
# Combine both sets into a new set
combined_set = set_a.union(set_b)
print(combined_set)
```

Output Block

```
{1, 2, 3, 'c', 'a', 'b'}
```

You can use the `|` operator instead of the `union()` method, and you will get the same result.

```
# Use | to join two sets.
set1 = {"a", "b", "c"}
set2 = {1, 2, 3}
# Using the pipe operator for union
set3 = set1 | set2
print(set3)
```

Output Block

```
{1, 2, 3, 'c', 'a', 'b'}
```

Join Multiple Sets

All the joining methods and operators can be used to join multiple sets.

When using a method, just add more sets in the parentheses, separated by commas.

```
set_a = {"a", "b", "c"}
set_b = {1, 2, 3}
set_c = {"John", "Elena"}
set_d = {"apple", "bananas", "cherry"}

# Join multiple sets into one set
merged_set = set_a.union(set_b, set_c, set_d)

print(merged_set)
```

Output Block

```
{1, 2, 3, 'Elena', 'John', 'c', 'cherry', 'bananas', 'a', 'apple', 'b'}
```

```
# Use | to join two sets:
set1 = {"a", "b", "c"}
set2 = {1, 2, 3}
set3 = {"John", "Elena"}
set4 = {"apple", "bananas", "cherry"}
# Using multiple '|' operator for the union operation
myset = set1 | set2 | set3 | set4
print(myset)
```

Output Block

```
{1, 2, 3, 'Elena', 'John', 'c', 'cherry', 'bananas', 'a', 'apple', 'b'}
```

Join a Set and a Tuple

The `union()` method allows you to join a set with other data types, like lists or tuples.

The result will be a set.

```
# Join a set with a tuple:

x = {"a", "b", "c"}
y = (1, 2, 3)
# using a union function to achieve the same between a set and a tuple
z = x.union(y)
print(z)
```

Output Block

```
{1, 2, 3, 'c', 'a', 'b'}
```

Update

- The `update()` method inserts all items from one set into another.
- The `update()` changes the original set, and does not return a new set.

```
# The update() method inserts the items in set2 into set1:
set1 = {"a", "b", "c"}
set2 = {1, 2, 3}
# Using an update method
set1.update(set2)
print(set1)
```

Output Block

```
{1, 2, 3, 'c', 'a', 'b'}
```

Intersection

Keep ONLY the duplicates

The `intersection()` method will return a new set, that only contains the items that are present in both sets.

```
# Join set1 and set2, but keep only the duplicates

set1 = {"apple", "banana", "cherry"}
set2 = {"google", "microsoft", "apple"}

# To keep the common elements from two sets
set3 = set1.intersection(set2)
print(set3)
```

Output Block

```
{'apple'}
```

You can use the `&` operator instead of the `intersection()` method, and you will get the same result.

```
set1 = {"apple", "banana", "cherry"}
set2 = {"google", "microsoft", "apple"}

# Use `&` to join two sets:
set3 = set1 & set2
print(set3)
```

Output Block

```
{'apple'}
```

Note: The `&` operator can only be used to find the intersection between sets. Unlike the `intersection()` method, it cannot be directly applied with other data types.

The `intersection_update()` method also retains only the common elements between sets, but instead of returning a new set, it updates the original set in place.

```
# Keep the items that exist in both set1, and set2:

set_a = {"apple", "banana", "cherry"}
set_b = {"google", "microsoft", "apple"}

# Keep only items that exist in both sets using intersection_update
set_a.intersection_update(set_b)

print(set_a)
```

The values `True` and `1` are considered the same value. The same goes for `False` and `0`.

```
# Join sets that contains the values True, False, 1, and 0, and see what is considered
as duplicates:
set1 = {"apple", 1, "banana", 0, "cherry"}
set2 = {False, "google", 1, "apple", 2, True}
set3 = set1.intersection(set2)
print(set3)
```

Output Block

```
{False, 1, 'apple'}
```

Difference

The `difference()` method will return a new set that will contain only the items from the first set that are not present in the other set.

```
set1 = {"apple", "banana", "cherry"}
set2 = {"google", "microsoft", "apple"}
# Keep all items from set1 that are not in set2:
```

```
set3 = set1.difference(set2)
print(set3)
```

Output Block

```
{'cherry', 'banana'}
```

You can use the - operator instead of the difference() method, and you will get the same result.

```
# Use - to join two sets.
set3 = set1 - set2
print(set3)
```

Output Block

```
{'cherry', 'banana'}
```

The `difference_update()` method will keep the items from the first set that are not in the other set, but it will change the original set instead of returning a new set.

```
set1 = {"apple", "banana", "cherry"}
set2 = {"google", "microsoft", "apple"}
# Use the difference_update() method to keep only the items from the first set that
# are not present in the other set.
set1.difference_update(set2)
print(set1)
```

Output Block

```
{'cherry', 'banana'}
```

Symmetric Differences

The `symmetric_difference()` method will keep only the elements that are NOT present in both sets.

```
set1 = {"apple", "banana", "cherry"}
set2 = {"google", "microsoft", "apple"}
# Keep the items that are not present in both sets.
set3 = set1.symmetric_difference(set2)
print(set3)
```

Output Block

```
{'google', 'microsoft', 'banana', 'cherry'}
```

You can use the `^` operator instead of the `symmetric_difference()` method, and you will get the same result.

```
# Use ^ to join two sets.
set3 = set1 ^ set2
print(set3)
```

Output Block

```
{'google', 'microsoft', 'banana', 'cherry'}
```

The `symmetric_difference_update()` method will also keep all but the duplicates, but it will change the original set instead of returning a new set.

```
# symmetric_difference_update() method to keep the items that are not present in both
sets

set1 = {"apple", "banana", "cherry"}
set2 = {"google", "microsoft", "apple"}

set1.symmetric_difference_update(set2)

print(set1)
```

Output Block

```
{'google', 'microsoft', 'banana', 'cherry'}
```

Python frozenset

A `frozenset` is an immutable version of a set in Python.

Like a set, it stores unique, unordered elements that cannot be changed.

However, unlike a regular set, a `frozenset` does not allow adding or removing elements after it is created.

Creating a frozenset

You can create a `frozenset` using the `frozenset()` constructor with any iterable as input.

```
# Create a frozenset and check its type:

x = frozenset({"apple", "banana", "cherry"})
print(x)
print(type(x))
```

Output Block

```
frozenset({'cherry', 'banana', 'apple'})
<class 'frozenset'>
```

Frozenset Methods

Since frozensets are immutable, you cannot add or remove elements. However, they still support all non-modifying set operations.

Method	Shortcut	Description
<code>copy()</code>		Returns a shallow copy of the frozenset
<code>difference()</code>	-	Returns a new frozenset containing elements that are not in the other set
<code>intersection()</code>	&	Returns a new frozenset with common elements
<code>isdisjoint()</code>		Returns <code>True</code> if two frozensets have no elements in common
<code>issubset()</code>	<code><=</code> / <code><</code>	Returns <code>True</code> if the frozenset is a subset (or proper subset) of another
<code>issuperset()</code>	<code>>=</code> / <code>></code>	Returns <code>True</code> if the frozenset is a superset (or proper superset) of another
<code>symmetric_difference()</code>	<code>^</code>	Returns a new frozenset with elements present in either set, but not both
<code>union()</code>	pipe_symbol	Returns a new frozenset containing all elements from both sets

Python - Set Methods

Python provides a variety of built-in methods for working with sets.

Method	Shortcut	Description
<code>add()</code>		Adds an element to the set
<code>clear()</code>		Removes all elements from the set
<code>copy()</code>		Returns a shallow copy of the set
<code>difference()</code>	<code>-</code>	Returns a set containing elements present in one set but not in others
<code>difference_update()</code>	<code>-=</code>	Removes elements from this set that are also present in another set
<code>discard()</code>		Removes a specified element if it exists
<code>intersection()</code>	<code>&</code>	Returns a set containing only common elements
<code>intersection_update()</code>	<code>&=</code>	Updates the set by keeping only elements found in other set(s)
<code>isdisjoint()</code>		Returns <code>True</code> if two sets have no elements in common
<code>issubset()</code>	<code><=</code>	Returns <code>True</code> if all elements of this set are in another set
<code>issubset()</code>	<code><</code>	Returns <code>True</code> if this set is a proper subset of another larger set
<code>issuperset()</code>	<code>>=</code>	Returns <code>True</code> if this set contains all elements of another set
<code>issuperset()</code>	<code>></code>	Returns <code>True</code> if this set is a proper superset of another smaller set
<code>pop()</code>		Removes and returns an arbitrary element from the set
<code>remove()</code>		Removes a specified element from the set
<code>symmetric_difference()</code>	<code>^</code>	Returns a set with elements present in either set, but not in both
<code>symmetric_difference_update()</code>	<code>^=</code>	Updates the set with the symmetric difference
<code>union()</code>	<code>,</code>	<code>,</code>
<code>update()</code>	<code>,</code>	<code>=,</code>

Python Dictionaries

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}
```

Dictionary

Dictionaries are used to store data values in key:value pairs.

A dictionary is a collection that is ordered*, changeable, and does not allow duplicate keys.

```
# Create and print a dictionary:  
  
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
} # Created a dictionary with the details of a car like (brand, model and year)  
print(thisdict)
```

Output Block

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}
```

Dictionary Items

Dictionary items are ordered, changeable, and do not allow duplicate keys.

Dictionary items are stored in key:value pairs and can be accessed using the key name.

```
# to print only the brand(one key's value) from the dictionary  
  
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
print(thisdict["brand"])
```

Output Block

```
Ford
```

Ordered or Unordered?

When we say dictionaries are ordered, it means the items have a defined sequence, and that sequence will not change.

Unordered means that the items do not have a defined sequence, and you cannot access items using an index.

Changeable

Dictionaries are changeable, meaning that we can change, add or remove items after the dictionary has been created.

Duplicates Not Allowed Dictionaries cannot have two items with the same key.

```
# Duplicate values will overwrite existing values:
```

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964,  
    "year": 2020  
} # Here the year is entered twice in the dictionary, the Value of year is overwritten  
based on the latest update to the key.  
print(thisdict)
```

Output Block

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 2020}
```

```
# To determine how many items a dictionary has, use the len() function  
print(len(thisdict))
```

Output Block

```
3
```

Dictionary Items - Data Types

The values in dictionary items can be of any data type, meaning a dictionary can store integers, strings, floats, booleans, lists, or even other dictionaries as values.

```
# String, int, boolean, and list data types in a single dict variable
```

```
thisdict = {  
    "brand": "Ford",  
    "electric": False,  
    "year": 1964,  
    "colors": ["red", "white", "blue"]  
}  
  
print(type(thisdict)) # returns the type belonging to the class dict
```

Output Block

```
<class 'dict'>
```

The dict() Constructor

It is also possible to create a dictionary using the `dict()` constructor.

The `dict()` constructor provides an alternative way to initialize a dictionary from key-value pairs or other mapping data.

```
# Using the dict() method to make a dictionary

thisdict = dict(name = "John", age = 36, country = "Norway")
print(thisdict)
```

Output Block

```
{'name': 'John', 'age': 36, 'country': 'Norway'}
```

Accessing Items

You can access the items of a dictionary by referring to its key name, inside square brackets.

```
# Get the value of the "model" key:
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
x = thisdict["model"] #Returns the value associated to the key 'model'
print(x)

# There is also a method called get() that will give you the same result.
x = thisdict.get("model")
print(x)
```

Output Block

```
Mustang
Mustang
```

Get Keys

The `keys()` method returns a view object containing all the keys in the dictionary.

```
# Returns all the available keys present in the dictionary variable
x = thisdict.keys()
print(x)
```

Output Block

```
dict_keys(['brand', 'model', 'year'])
```

The list of the keys is a view of the dictionary, meaning that any changes done to the dictionary will be reflected in the keys list.

```
# Adding a new item to the original dictionary.
car = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}

x = car.keys() # to display all the keys before adding a new key

print(x) #before the change

car["color"] = "white"

print(x) #after the change
```

Output Block

```
dict_keys(['brand', 'model', 'year'])
dict_keys(['brand', 'model', 'year', 'color'])
```

Get Values

The `values()` method will return a list of all the values in the dictionary.

```
# Get a list of the values:
x = thisdict.values()
print(x)
```

Output Block

```
dict_values(['Ford', 'Mustang', 1964])
```

The list of the values is a view of the dictionary, meaning that any changes done to the dictionary will be reflected in the values list.

```
# Make a change in the original dictionary, and see that the values list gets updated
as well:

car = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}

x = car.values() # List the values in the dictionary

print(x) #before the change
```

```
car["year"] = 2020

print(x) #after the change
```

Output Block

```
dict_values(['Ford', 'Mustang', 1964])
dict_values(['Ford', 'Mustang', 2020])
```

```
# Add a new item to the original dictionary.
car = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}

x = car.values()

print(x) #before the change

car["color"] = "red"

print(x) #after the change
```

Output Block

```
dict_values(['Ford', 'Mustang', 1964])
dict_values(['Ford', 'Mustang', 1964, 'red'])
```

Get Items

The `items()` method will return each item in a dictionary, as tuples in a list.

```
# Get a list of the key:value pairs of the dictionary
x = thisdict.items()
print(x)
```

Output Block

```
dict_items([('brand', 'Ford'), ('model', 'Mustang'), ('year', 1964)])
```

The returned list is a view of the items of the dictionary, meaning that any changes done to the dictionary will be reflected in the items list.

```
# Make a change in the original dictionary.
car = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
```

```

}

x = car.items() # Items have been updated

print(x) #before the change

car["year"] = 2020

print(x) #after the change

```

Output Block

```

dict_items([('brand', 'Ford'), ('model', 'Mustang'), ('year', 1964)])
dict_items([('brand', 'Ford'), ('model', 'Mustang'), ('year', 2020)])

```

Check if Key Exists

To determine if a specified key is present in a dictionary use the `in` keyword.

```

# Check if "model" is present in the dictionary:
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
if "model" in thisdict: # Using the membership operator we check if the key is present
in the dictionary
    print("Yes, 'model' is one of the keys in the thisdict dictionary")

```

Output Block

```

Yes, 'model' is one of the keys in the thisdict dictionary

```

Change Values

You can change the value of a specific item by referring to its key name.

```

# Change the "year" to 2018:

thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
print("Before",thisdict["year"])
thisdict["year"] = 2018 # The value of the year is overwritten to 2018
print("After",thisdict["year"])

```

Output Block

Before 1964
After 2018

Update Dictionary

The `update()` method updates the dictionary with the items from the given argument.

The argument must be another dictionary or an iterable containing key:value pairs.

```
# Update the "year" of the car by using the update() method.

thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
print("Before",thisdict["year"])
thisdict.update({"year": 2020}) # Update the year to 2020
print("After",thisdict["year"])
```

Output Block

Before 1964
After 2020

Adding Items

Adding an item to the dictionary is done by using a new index key and assigning a value to it.

```
# thisdict variable initialized with the car details
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
# Adding color key and value for the dictionary
thisdict["color"] = "red"
print(thisdict)
```

Output Block

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964, 'color': 'red'}
```

Update Dictionary

The `update()` method will update the dictionary with the items from a given argument. If the item does not exist, the item will be added.

The argument must be a dictionary, or an iterable object with key:value pairs.

```
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
# Adding the key and value of color to dictionary with the update operation
thisdict.update({"color": "red"})
```

Python - Remove Dictionary Items

Removing Items

There are multiple ways to remove items from a dictionary in Python.

```
# Dictionary of a car detail
car_info = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}

# Remove the item with key "model"
car_info.pop("model")
print(car_info)
```

Output Block

```
{'brand': 'Ford', 'year': 1964}
```

The `popitem()` method removes the last inserted item (in versions before 3.7, a random item is removed instead).

```
# Remove the last inserted item from the dictionary
car_info.popitem()
print(car_info)
```

Output Block

```
{'brand': 'Ford'}
```

```
# Dictionary of a car detail
car_info = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}

print(car_info) # Before
# Remove the item with key "model" using del keyword
```

```
del car_info["model"]

print(car_info) # After
```

Output Block

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}
{'brand': 'Ford', 'year': 1964}
```

```
# Dictionary of a car detail
car_info = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}

# Delete the entire dictionary
del car_info

# This will raise an error because the dictionary no longer exists
print(car_info)
```

Output Block

```
NameError: name 'car_info' is not defined
```

```
# Dictionary of a car detail
car_info = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}

# Remove all items from the dictionary
car_info.clear()

print(car_info) # Returns an empty dictionary
```

Output Block

```
{}
```

Python - Loop Dictionaries

Loop Through a Dictionary

You can iterate through a dictionary using a **for** loop.

By default, looping through a dictionary returns its keys. However, there are methods available to access values or key-value pairs as well.

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}
```

```
# Print all key names in the dictionary, one by one.  
for x in thisdict:  
    print(x)
```

Output Block

```
brand  
model  
year
```

```
# Print all values in the dictionary, one by one.  
for x in thisdict:  
    print(thisdict[x])
```

Output Block

```
Ford  
Mustang  
1964
```

```
# You can also use the values() method to return values of a dictionary.  
for x in thisdict.values():  
    print(x)
```

Output Block

```
Ford  
Mustang  
1964
```

```
# You can use the keys() method to return the keys of a dictionary.  
for x in thisdict.keys():  
    print(x)
```

Output Block

```
brand  
model  
year
```

```
# Loop through both keys and values, by using the items() method.
for x, y in thisdict.items():
    print(x, y)
```

Output Block

```
brand Ford
model Mustang
year 1964
```

Python - Copy Dictionaries

Copy a Dictionary

You cannot copy a dictionary by simply assigning it using `dict2 = dict1`, because this only creates a reference to the original dictionary. Any changes made to `dict1` will also affect `dict2`.

To properly copy a dictionary, you can use built-in methods such as the `copy()` method.

```
# Make a copy of a dictionary with the copy() method.

thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
# using copy() method
mydict = thisdict.copy()
print(mydict)
```

Output Block

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}
```

Another way to make a copy is to use the built-in function `dict()`.

```
# Make a copy of a dictionary with the dict() function:
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
# Using a dict() method to create a new copy of the existing dictionary
mydict = dict(thisdict)
print(mydict)
```

Output Block

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}
```

Nested Dictionaries

A dictionary can hold other dictionaries as values. This structure is known as a **nested dictionary**.

```
# Create a dictionary that contain three dictionaries

# A dictionary called myfamily containing nested dictionaries
# Each key (child1, child2, child3) represents a child

myfamily = {
    "child1" : {
        "name" : "Emil",
        "year" : 2004
    },
    "child2" : {
        "name" : "Tobias",
        "year" : 2007
    },
    "child3" : {
        "name" : "Linus",
        "year" : 2011
    }
}
```

Or, if you want to add three dictionaries into a new dictionary.

```
# Create three dictionaries, then create one dictionary that will contain the other
three dictionaries.

child1 = {
    "name" : "Emil",
    "year" : 2004
}
child2 = {
    "name" : "Tobias",
    "year" : 2007
}
child3 = {
    "name" : "Linus",
    "year" : 2011
}

myfamily = {
    "child1" : child1,
    "child2" : child2,
    "child3" : child3
}
```

Access Items in Nested Dictionaries

To access items from a nested dictionary, you use the name of the dictionaries, starting with the outer dictionary.

```
# Print the name of child 2
print(myfamily["child2"]["name"])
```

Output Block

Tobias

Loop Through Nested Dictionaries

You can iterate through a nested dictionary using the `items()` method.

This allows you to access both the keys and their corresponding values while looping through the dictionary.

```
# Loop through the keys and values of all nested dictionaries.

for x, obj in myfamily.items():
    print(x)

    for y in obj:
        print(y + ': ', obj[y])
```

Output Block

```
child1
name: Emil
year: 2004
child2
name: Tobias
year: 2007
child3
name: Linus
year: 2011
```

Python Dictionary Methods

Python provides several built-in methods for working with dictionaries.

Method	Description
<code>clear()</code>	Removes all elements from the dictionary
<code>copy()</code>	Returns a shallow copy of the dictionary
<code>fromkeys()</code>	Creates a new dictionary with specified keys and a given value
<code>get()</code>	Returns the value of a specified key
<code>items()</code>	Returns a view of key-value pairs as tuples
<code>keys()</code>	Returns a view of all keys in the dictionary
<code>pop()</code>	Removes the element with the specified key
<code>popitem()</code>	Removes the last inserted key-value pair
<code>setdefault()</code>	Returns the value of a key; if not present, inserts it with a specified value
<code>update()</code>	Updates the dictionary with specified key-value pairs
<code>values()</code>	Returns a view of all values in the dictionary